

# CS 486/686

# Training Models From Scratch

Yuntian Deng

---

Lecture 17

Core loop: data  $\rightarrow$  model  $\rightarrow$  loss  $\rightarrow$  gradient  $\rightarrow$  update

# Recorded lecture (asynchronous)



This lecture is **pre-recorded** - I'm away at **ICML** this week, so there's no in-person class today. Watch at your own pace.



**Due today (Tue Jul 7):** Chat 8. The CS686 project proposal is now due **Thu Jul 9** (submit on LEARN under "Project Proposal").



Questions? Post on **Piazza** - the TAs are available, and I'll follow up when I'm back.

# Today's target

By the end, this loop should feel readable:

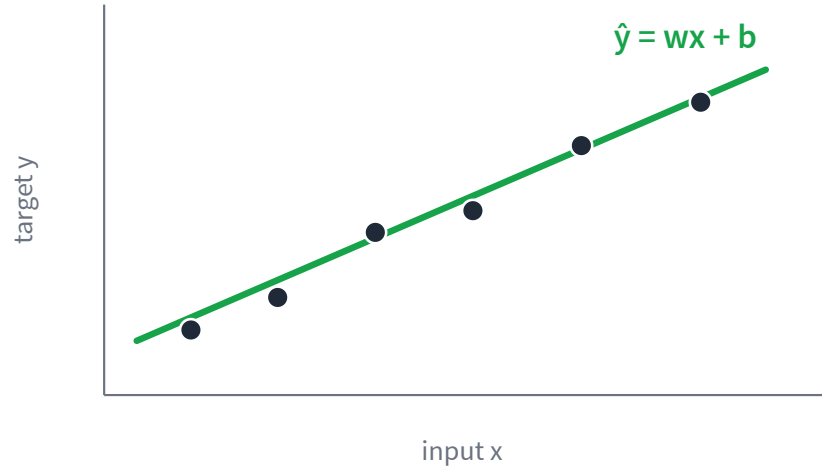
```
for x, y in loader:  
    pred = model(x)  
    loss = loss_fn(pred, y)  
    loss.backward()  
    optimizer.step()  
    optimizer.zero_grad()
```

Everything later - neural nets, LLMs, diffusion - scales up this idea.

# The learning recipe



# Parameters are the knobs

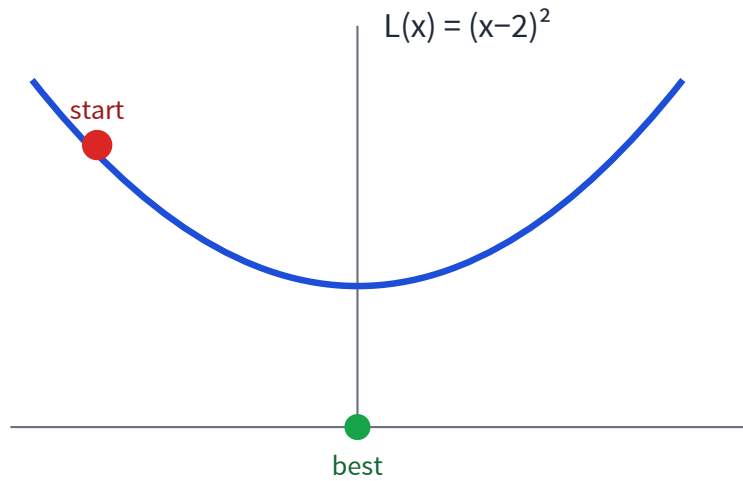


The data is fixed.

The model has tunable **parameters**:  $w$  and  $b$ .

Training means choosing parameters that make predictions good.

# Start with one knob



Forget ML for one slide.

We only need to make a loss small.

This is the smallest possible version of training.

# Move downhill

The gradient points uphill, so step the other way.

$$x \leftarrow x - \eta \frac{dL}{dx}$$

$$-dL/dx$$

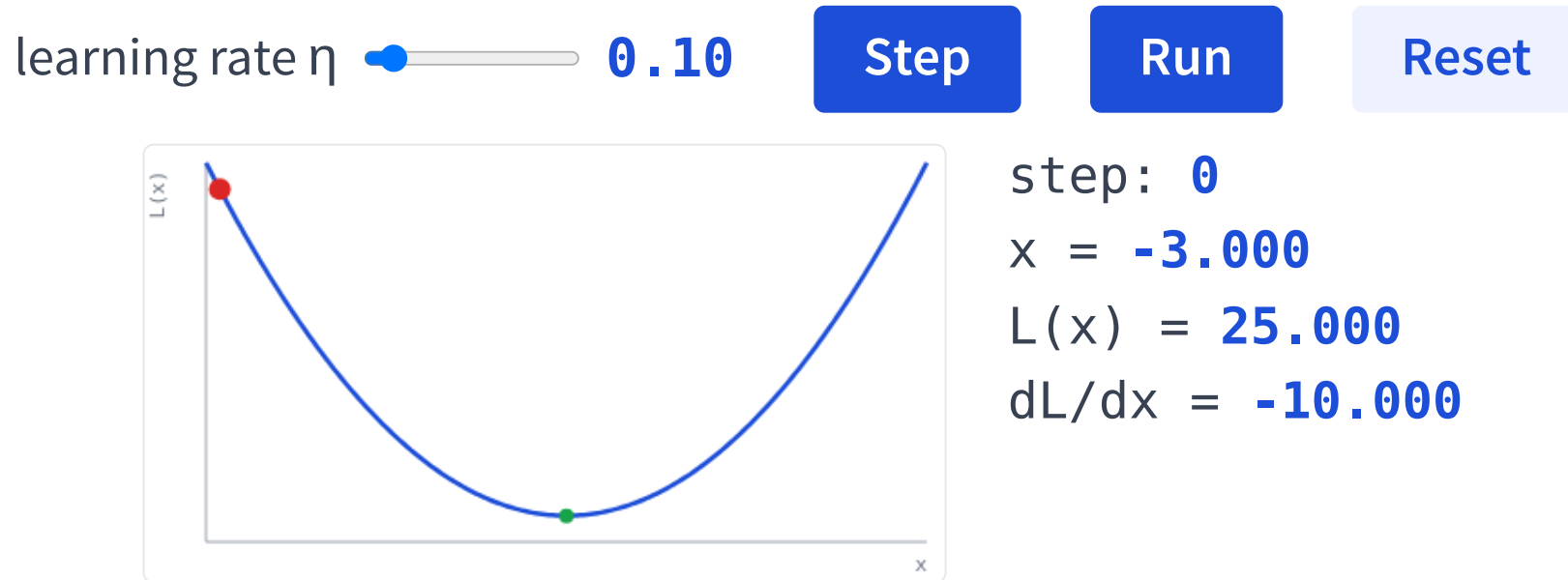
direction that decreases loss

$$\eta$$

learning rate: how big a step?

# Playground: gradient descent LIVE

Interactive demo — runs live in your browser (open the slides to try it).



Small  $\eta$ : slow but steady. Around  $\eta = 1$  it overshoots; push higher and it diverges.



# PyTorch computes gradients for us

## Manual derivative

```
def loss(x):  
    return (x - 2) ** 2  
  
def manual_grad(x):  
    return 2 * (x - 2)
```

## Automatic differentiation

```
x = nn.Parameter(torch.tensor(-4.0))  
L = (x - 2) ** 2  
L.backward()  
print(x.grad)  # -12
```

For millions of parameters, we rely on autograd. But how does it know the answer?

# How can gradients be automatic?

Every operation records its own **local derivative**; the **chain rule** multiplies them backward.

Forward:  $u = x - 2$ , then  $L = u^2$ .

Chain rule:  $\frac{dL}{dx} = \frac{dL}{du} \cdot \frac{du}{dx}$

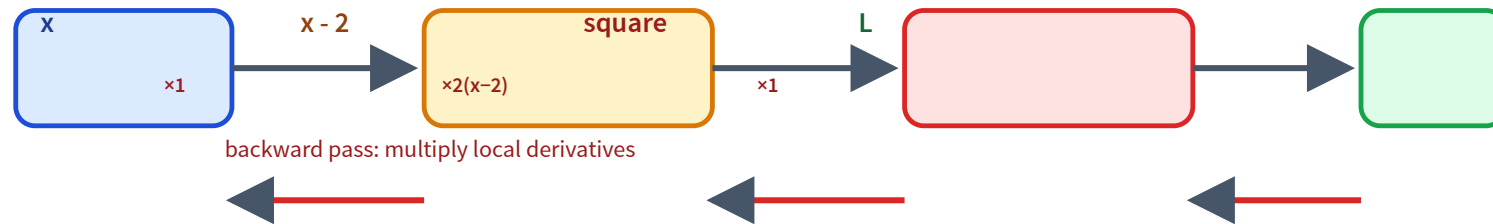
Local:  $\frac{dL}{du} = 2u$  (since  $L = u^2$ )

Local:  $\frac{du}{dx} = 1$  (since  $u = x - 2$ )

Multiply:  $\frac{dL}{dx} = 2u \cdot 1 = 2(x - 2)$

At  $x = -4$ :  $\frac{dL}{dx} = 2(-6) = -12$  — exactly x.grad.

# Autograd follows the computation graph



# Run it: autograd from scratch

[LIVE PYTHON](#)

Interactive demo — runs live in your browser (open the slides to try it).

[Run Python](#)[Reset code](#)

```
# A tiny reverse-mode autograd (this is what .b
class Value:
    def __init__(self, data, _children=()):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)

    def __sub__(self, other):
        other = other if isinstance(other, Value) else Value(other.data)
        out = Value(self.data - other.data, (self, other))
        def _backward():
            self.grad -= out.grad
            other.grad += out.grad
```

Click "Run Python" to execute.

Real Python in your browser. The graph  $x \rightarrow (x-2) \rightarrow \text{square} \rightarrow L$  is built as you compute, then `backward()` multiplies local derivatives.

# From one knob to many

A model has many parameters, collected as  $\theta$ . The loop does not change.

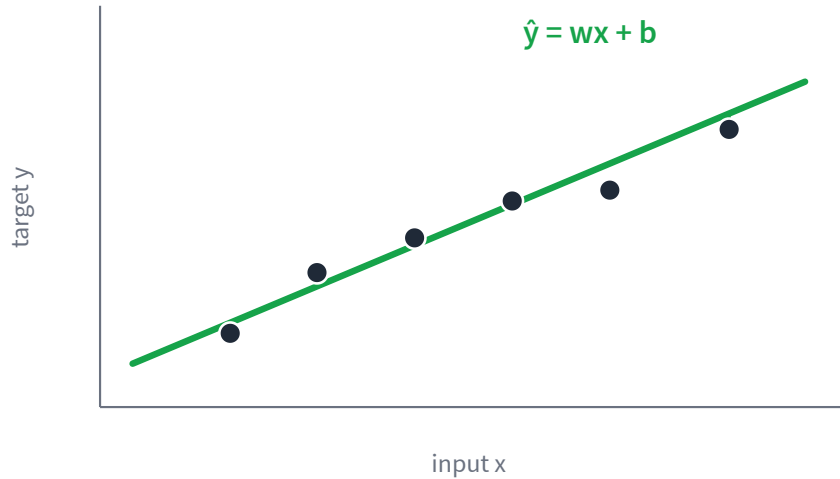
$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

```
optimizer = SGD(model.parameters(), lr=0.1)

for x, y in loader:
    pred = model(x)
    loss = loss_fn(pred, y)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

`model.parameters()` → the parameters  $\theta$   
`loss.backward()` → the gradient  $\nabla_{\theta} L$   
`optimizer.step()` → update  $\theta \leftarrow \theta - \eta \nabla_{\theta} L$

# Example 1: regression



Prediction:  $\hat{y} = wx + b$  ( $w, b$  are the parameters)

$$\text{Error: } L = \frac{1}{m} \sum_i (\hat{y}_i - y_i)^2$$

Start with a bad line — big errors (dashed).

Measure the error, nudge  $w, b$  to shrink it.

Repeat — the errors keep shrinking.

Good fit: the errors are small.

# Regression in PyTorch

```
# the knob: a line  $y = wx + b$ 
```

```
model = nn.Linear(1, 1)
```

```
# how we score error: mean squared error
```

```
loss_fn = nn.MSELoss()
```

```
# how we update the parameters
```

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
```

```
# forward: predict from inputs
```

```
pred = model(x)
```

```
# measure how wrong we are
```

```
loss = loss_fn(pred, y)
```

```
# autograd fills in the gradients
```

```
loss.backward()
```

```
# one step downhill: update w and b
```

```
optimizer.step()
```

```
# clear gradients before the next batch
```

```
optimizer.zero_grad()
```

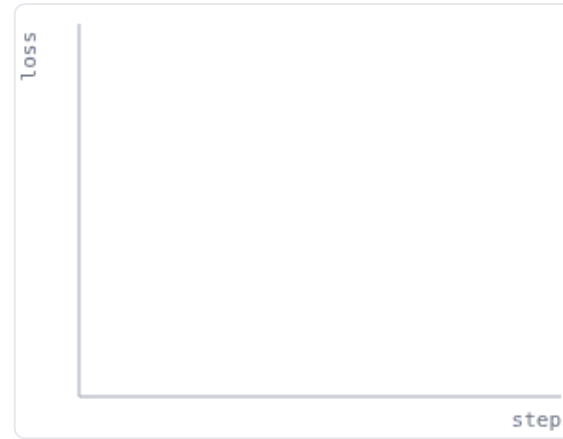
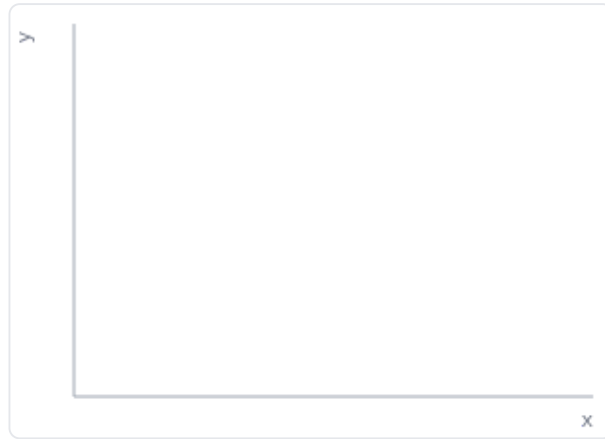
# Run it: train the line

[LIVE PYTHON](#)

Interactive demo — runs live in your browser (open the slides to try it).

learning rate  **0.10** steps  **60**

**Train**



Click "Train" to run the loop in Python.

Same loop as the slide, in numpy: predict  $\rightarrow$  MSE  $\rightarrow$  gradients  $\rightarrow$  update. Watch the line snap to the data as the loss drops. High learning rate can overshoot.



## Example 2: classification

Now the target is a **category**, not a number.

### Regression

$$\hat{y} = 217.3$$

a number

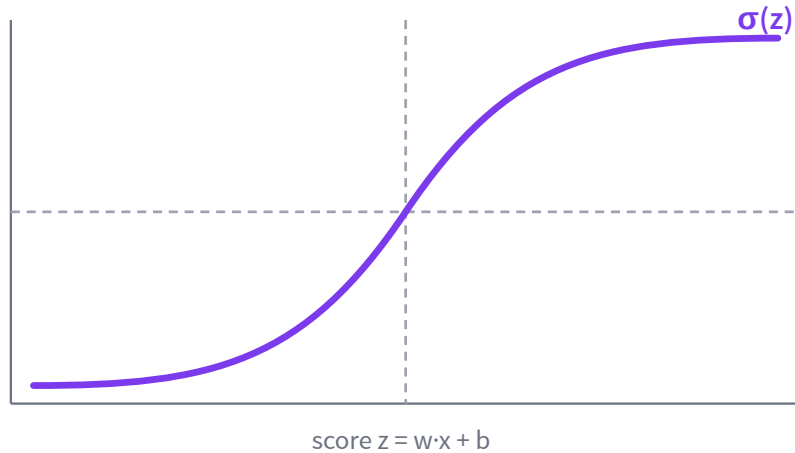
### Classification

$$\hat{y} = \text{“spam”}$$

a class

But gradient descent needs a smooth output — so we predict a **probability** instead.

# Logistic regression: predict a probability



Compute a score:

$$z = w \cdot x + b$$

Squash it into a probability:

$$P(\text{spam} \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

# Loss for classification

Cross-entropy punishes being confidently wrong.

Recall  $\hat{y} = P(y = 1 \mid x) = \sigma(z)$  – the probability the model predicts.

$$L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

True label spam  $\Rightarrow y = 1$ , so  $L = -\log \hat{y}$ .

**confident & right**

$$\hat{y} = 0.9$$

$$L = -\log 0.9 \approx 0.11$$

**confident & wrong**

$$\hat{y} = 0.1$$

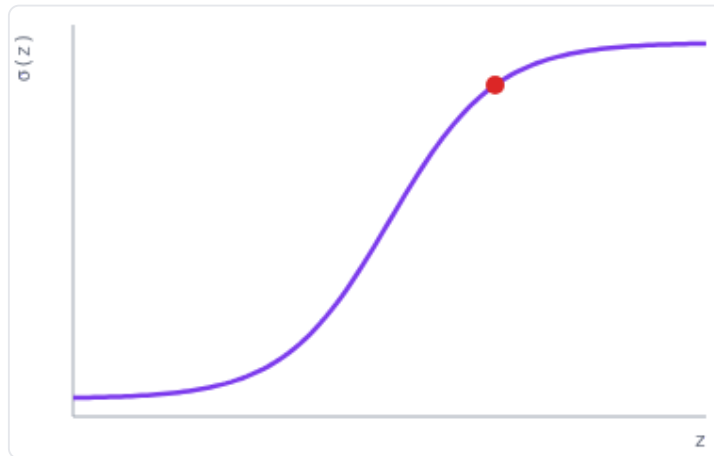
$$L = -\log 0.1 \approx 2.30$$

# Playground: cross-entropy loss LIVE

Interactive demo — runs live in your browser (open the slides to try it).

score  $z$   **2.0**

**flip true label**



true label: **spam (y=1)**

$\hat{y} = \sigma(z) =$  **0.881**

loss = **0.127**

**confident & right**

Cross-entropy =  $-\log(\text{prob of the true class})$ . Confident and wrong is punished hard.

# Many classes: softmax

One score per class; softmax turns scores into probabilities.

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$



Next-token prediction is exactly this, over a vocabulary.

# Playground: softmax & temperature LIVE

Interactive demo — runs live in your browser (open the slides to try it).

temperature  1.0

dog  1.0

cat  3.0

bird  1.5

fish  0.5



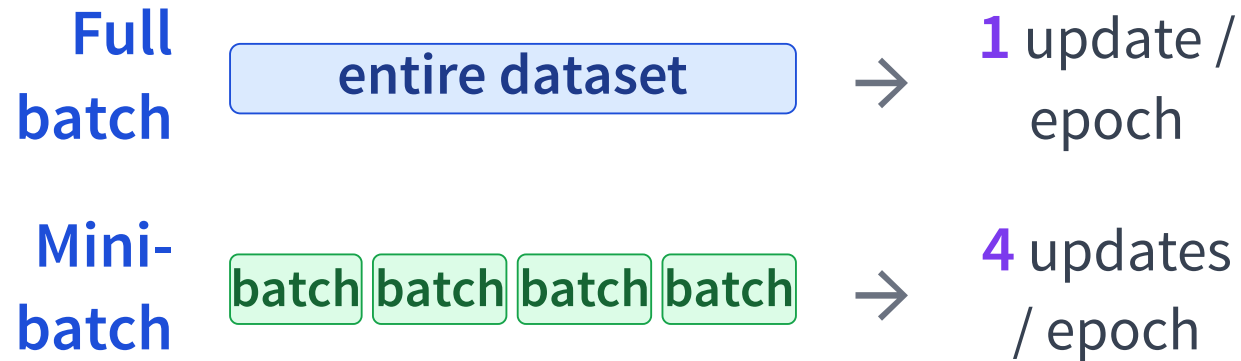
Softmax turns scores into probabilities. Low temperature sharpens; high temperature flattens (the decoding knob in L20/L21).

# The full loop, annotated

```
# loop over the training data
for x, y in loader:
    # forward pass: predict
    pred = model(x)
    # measure the error
    loss = loss_fn(pred, y)
    # backward pass: compute gradients
    loss.backward()
    # update the parameters
    optimizer.step()
    # reset gradients for the next step
    optimizer.zero_grad()
```

This is the pattern to recognize for the rest of the module.

# Why mini-batches?



Full batch: few, expensive, smooth steps. Mini-batch: many, cheap, noisy steps — usually faster progress, and GPU-friendly.



# Split the data: train / validation / test

## train

updates the weights

## validation

chooses settings

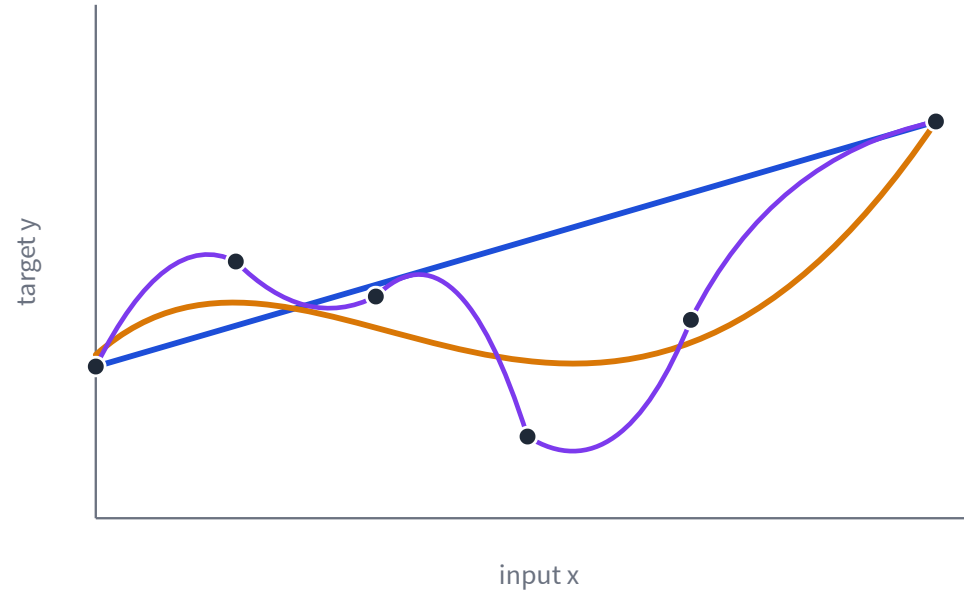
## test

final, one-time  
estimate

Never tune on the test set.

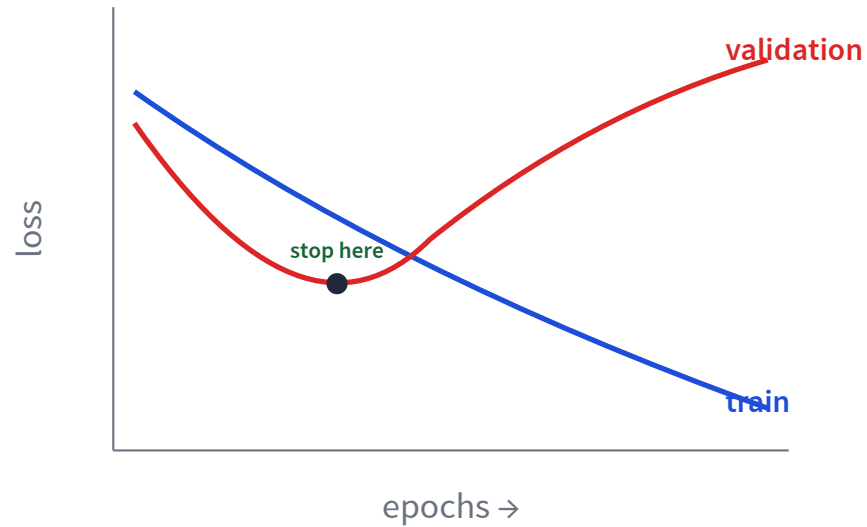
# More power is not always better

Same data, increasingly flexible models.



— underfit    — good fit    — overfit

# Watch train and validation loss



Training loss keeps dropping.

Validation loss starts **rising** — that growing gap is overfitting. Stop where validation bottoms out.

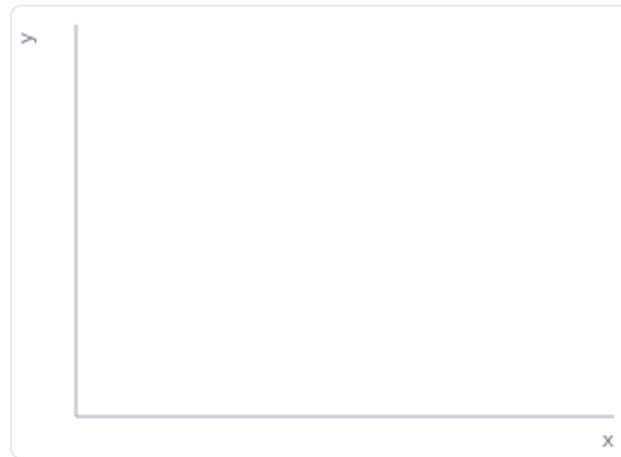
# Playground: overfitting

[LIVE PYTHON](#)

Interactive demo — runs live in your browser (open the slides to try it).

polynomial degree  1

Fit in Python



Blue = training points, red = validation points. Raise the degree: training loss keeps dropping but validation loss turns up — that gap is overfitting.

# Three ways to fight overfitting

## More data

show the model  
more of the world

## Regularize

prefer simpler  
weights

$$L(\theta) + \lambda \|\theta\|^2$$

## Early stop

stop when validation  
turns up

# What carries forward?

Only the **model** and the **loss** change. The loop stays the same.

| Task               | Model (architecture) | Loss                          |
|--------------------|----------------------|-------------------------------|
| Regression (today) | Linear / MLP         | squared error                 |
| Language model     | Transformer          | cross-entropy (next token)    |
| Image generation   | Diffusion U-Net      | squared error (predict noise) |

forward → loss → backward → step, every time.

## Recap - next: learned representations

- ✓ A model learns by minimizing a **loss**.
- ✓ **Autograd** computes gradients through a graph.
- ✓ Regression uses squared error; classification uses **cross-entropy**.
- ✓ Validation loss catches **overfitting**.

L18: if linear models use fixed features, how do neural nets learn their own?